Quantum Optimization Practical Session

Travelling Salesman Problem & 0/1 Knapsack Problem

Classical Algorithms -> QUBO/Ising Formulation -> QAOA (Qiskit)

Complete build-ready material: full source code, executed outputs, diagrams and comparison charts

Prepared for: First Quantum Optimization Practical Class
Toolkit: Python 3.12 · Qiskit 1.4 · qiskit-optimization 0.7 · qiskit-algorithms 0.4

0. Overview & Learning Objectives

This practical session covers two of the most widely used benchmark problems in combinatorial optimization the Travelling Salesman Problem (TSP) and the 0/1 Knapsack Problem. Both problems are solved end-to-end through the same pedagogical journey, connecting directly to later syllabus topics such as QAOA and Quantum Annealing.

The complete journey for each problem

1. Real World Problem
2. Problem Statement
3. Mathematical Model
4. Brute Force Solution
5. Better Classical Algorithm
6. Why Classical Struggles (scaling)
7. Quantum Formulation (QUBO / Ising Hamiltonian)
8. Quantum Implementation (QAOA in Qiskit)
9. Compare Results

Recommended Practical Structure

Step	Activity
1	Understand the real-world story
2	Write inputs and outputs
3	Draw the graph / table
4	Solve manually
5	Implement brute force in Python
6	Measure execution time
7	Implement a better classical algorithm
8	Compare the two classical methods
9	Introduce the quantum formulation (QUBO / Hamiltonian)
10	Run a simple Qiskit QAOA example (simulator) and compare results

Environment used to generate this material

- Python 3.12.3
- qiskit 1.4.4, qiskit-aer 0.16.4
- qiskit-optimization 0.7.0, qiskit-algorithms 0.4.0
- networkx 3.6.1, matplotlib 3.10.8, numpy 2.4.4

PART A - Travelling Salesman Problem (TSP)

A.1 Real-World Story

An Amazon delivery truck must visit five Karnataka cities - Bangalore, Mysore, Mangalore, Hubli and Belgaum -visiting every city exactly once and returning to the starting city. Which route is the shortest?

A.2 Problem as a Graph

The five cities and the road distances between them form a complete graph (K5). Every pair of cities is connected by an edge weighted by distance (km).

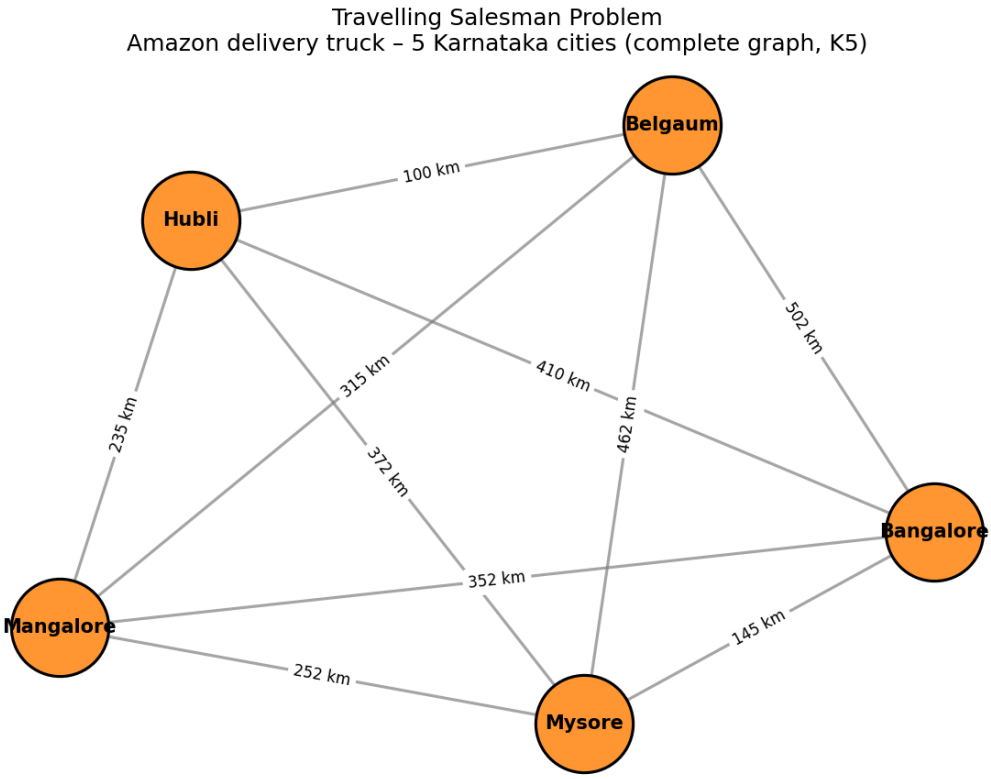

Figure A.1 — TSP modeled as a complete graph over 5 cities

How many possible routes?

For 4 cities: 24 total orderings (3 distinct routes once symmetry is removed). For 10 cities: 3,628,800 permutations. For 20 cities: computationally impossible to brute-force. This motivates the need for smarter algorithms.

A.3 Step 1 - Brute Force Solution

Generate every possible route, calculate its total distance, and keep the minimum. This guarantees the optimal answer but scales as O(n!).

Code: 01_tsp_bruteforce.py

```
"""
TSP - Step 2: Brute Force Solution
Generate EVERY possible route -> calculate distance -> choose minimum.
"""
import itertools
import time

cities = ['Bangalore', 'Mysore', 'Mangalore', 'Hubli', 'Belgaum']

distance = {
    ('Bangalore', 'Mysore'): 145,
    ('Bangalore', 'Mangalore'): 352,
    ('Bangalore', 'Hubli'): 410,
    ('Bangalore', 'Belgaum'): 502,
    ('Mysore', 'Mangalore'): 252,
    ('Mysore', 'Hubli'): 372,
    ('Mysore', 'Belgaum'): 462,
    ('Mangalore', 'Hubli'): 235,
    ('Mangalore', 'Belgaum'): 315,
    ('Hubli', 'Belgaum'): 100,
}

def get_distance(a, b):
    if a == b:
        return 0
    return distance.get((a, b), distance.get((b, a)))

def brute_force_tsp(cities):
    start = cities[0]
    remaining = cities[1:]

    best_route = None
    best_cost = float("inf")
    routes_checked = 0

    for perm in itertools.permutations(remaining):
        route = (start,) + perm
        total = 0
        for i in range(len(route) - 1):
            total += get_distance(route[i], route[i + 1])
        total += get_distance(route[-1], route[0])  # return to start
        routes_checked += 1

        if total < best_cost:
            best_cost = total
            best_route = route

    return best_route, best_cost, routes_checked

t0 = time.perf_counter()
best_route, best_cost, routes_checked = brute_force_tsp(cities)
t1 = time.perf_counter()

print("=== BRUTE FORCE TSP RESULT ===")
print(f"Routes evaluated : {routes_checked}")
print(f"Best route       : {' -> '.join(best_route)} -> {best_route[0]}")
print(f"Best distance    : {best_cost} km")
print(f"Time taken       : {(t1 - t0)*1000:.4f} ms")

# Scaling illustration (why brute force breaks down)
import math
print("\n=== WHY BRUTE FORCE DOESN'T SCALE ===")
for n in [4, 6, 8, 10, 12, 15, 20]:
    routes = math.factorial(n - 1) // 2
    print(f"{n:2d} cities -> {routes:,} distinct routes to check")

# Save results for later comparison
import json
with open('/home/claude/project/code/_tsp_bruteforce_result.json', 'w') as f:
```

```
    json.dump({
        "route": list(best_route),
        "cost": best_cost,
        "time_ms": (t1 - t0) * 1000,
        "routes_checked": routes_checked
    }, f, indent=2)
```

Executed Output

```
=== BRUTE FORCE TSP RESULT ===
Routes evaluated : 24
Best route       : Bangalore -> Mysore -> Mangalore -> Belgaum -> Hubli -> Bangalore
Best distance    : 1222 km
Time taken       : 0.0576 ms

=== WHY BRUTE FORCE DOESN'T SCALE ===
 4 cities -> 3 distinct routes to check
 6 cities -> 60 distinct routes to check
 8 cities -> 2,520 distinct routes to check
10 cities -> 181,440 distinct routes to check
12 cities -> 19,958,400 distinct routes to check
15 cities -> 43,589,145,600 distinct routes to check
20 cities -> 60,822,550,204,416,000 distinct routes to check
```

A.4 Step 2 — Better Classical Algorithm: Nearest Neighbor

Instead of checking every route, the Nearest Neighbor heuristic greedily moves to the closest unvisited city at each step. It runs in O(n²) time — dramatically faster than brute force — at the cost of not always finding the optimal route.

Code: 02_tsp_nearest_neighbor.py

```
"""
TSP - Step 3: Nearest Neighbor Heuristic (Better Classical Algorithm)
Greedy approach: always go to the closest unvisited city.
Much faster than brute force but not guaranteed to be optimal.
"""
import time

cities = ['Bangalore', 'Mysore', 'Mangalore', 'Hubli', 'Belgaum']

distance = {
    ('Bangalore', 'Mysore'): 145,
    ('Bangalore', 'Mangalore'): 352,
    ('Bangalore', 'Hubli'): 410,
    ('Bangalore', 'Belgaum'): 502,
    ('Mysore', 'Mangalore'): 252,
    ('Mysore', 'Hubli'): 372,
    ('Mysore', 'Belgaum'): 462,
    ('Mangalore', 'Hubli'): 235,
    ('Mangalore', 'Belgaum'): 315,
    ('Hubli', 'Belgaum'): 100,
}

def get_distance(a, b):
    if a == b:
        return 0
    return distance.get((a, b), distance.get((b, a)))

def nearest_neighbor_tsp(cities, start=None):
    start = start or cities[0]
    unvisited = set(cities)
    unvisited.remove(start)
    route = [start]
    current = start
    total_cost = 0
```

```
    while unvisited:
        nearest_city = min(unvisited, key=lambda city: get_distance(current, city))
        total_cost += get_distance(current, nearest_city)
        route.append(nearest_city)
        unvisited.remove(nearest_city)
        current = nearest_city

    total_cost += get_distance(current, start)  # return to start
    return route, total_cost

t0 = time.perf_counter()
nn_route, nn_cost = nearest_neighbor_tsp(cities)
t1 = time.perf_counter()

print("=== NEAREST NEIGHBOR TSP RESULT ===")
print(f"Route     : {' -> '.join(nn_route)} -> {nn_route[0]}")
print(f"Distance  : {nn_cost} km")
print(f"Time      : {(t1 - t0)*1000:.4f} ms")

# Compare against brute force optimal
import json
with open('/home/claude/project/code/_tsp_bruteforce_result.json') as f:
    bf = json.load(f)

gap = nn_cost - bf['cost']
gap_pct = (gap / bf['cost']) * 100

print("\n=== COMPARISON: BRUTE FORCE (OPTIMAL) vs NEAREST NEIGHBOR (HEURISTIC) ===")
print(f"{'Method':<20}{'Route Cost (km)':<18}{'Time (ms)':<12}")
print(f"{'Brute Force':<20}{bf['cost']:<18}{bf['time_ms']:<12.4f}")
print(f"{'Nearest Neighbor':<20}{nn_cost:<18}{(t1-t0)*1000:<12.4f}")
print(f"\nOptimality gap: {gap} km ({gap_pct:.1f}% worse than optimal)")

with open('/home/claude/project/code/_tsp_nn_result.json', 'w') as f:
    json.dump({"route": nn_route, "cost": nn_cost, "time_ms": (t1 - t0) * 1000}, f, indent=2)
```

Executed Output

```
=== NEAREST NEIGHBOR TSP RESULT ===
Route     : Bangalore -> Mysore -> Mangalore -> Hubli -> Belgaum -> Bangalore
Distance  : 1234 km
Time      : 0.0173 ms

=== COMPARISON: BRUTE FORCE (OPTIMAL) vs NEAREST NEIGHBOR (HEURISTIC) ===
Method              Route Cost (km)   Time (ms)
Brute Force         1222              0.0576
Nearest Neighbor    1234              0.0173

Optimality gap: 12 km (1.0% worse than optimal)
```

A.5 Step 3 — Quantum Formulation: TSP → QUBO → Hamiltonian → QAOA

Instead of checking every route, we encode the optimization problem into a cost Hamiltonian and use a parameterized quantum circuit (QAOA) to search for good solutions. QAOA is a hybrid quantum-classical algorithm: the quantum computer prepares candidate solutions while a classical optimizer updates the circuit parameters.

Workflow: TSP → Graph → QUBO → Ising Hamiltonian → QAOA → Optimal Route

Note on qubit count: the standard TSP QUBO encoding uses n² qubits for n cities. The full 5-city problem would need 25 qubits, which is heavy for a live classroom simulation. This material

uses a 4-city subset (16 qubits) for the live QAOA demo, while brute force and nearest neighbor use the full 5-city problem — this scaling trade-off is itself worth discussing with students.

Code: 03_tsp_qaoa.py

```
"""
TSP - Step 4: Quantum Formulation with QAOA (Qiskit)

Pipeline:  TSP -> Graph -> QUBO -> Ising Hamiltonian -> QAOA -> Optimal Route

QAOA is a hybrid quantum-classical algorithm:
  - The quantum computer prepares candidate solutions using a parameterized circuit.
  - A classical optimizer updates the circuit parameters to minimize the cost Hamiltonian.

NOTE: Because QAOA is simulated on a classical computer here (no real QPU access),
this is a full noiseless simulation useful for teaching the workflow. Even 4-5 city
TSP already needs many qubits (n^2 qubits for n cities using this encoding), so we
use a SMALL 4-city sub-problem for the live QAOA demo, and clearly explain why.
"""
import time
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt

from qiskit_optimization.applications import Tsp
from qiskit_optimization.converters import QuadraticProgramToQubo
from qiskit_algorithms import QAOA, NumPyMinimumEigensolver
from qiskit_algorithms.optimizers import COBYLA
from qiskit_algorithms.utils import algorithm_globals
from qiskit.primitives import StatevectorSampler as Sampler
from qiskit_optimization.algorithms import MinimumEigenOptimizer

algorithm_globals.random_seed = 42
np.random.seed(42)

# ---- Use a 4-city subset for the live QAOA demo ----------------------------
# n cities -> n^2 qubits under the standard QUBO encoding.
# 4 cities = 16 qubits (already heavy for a laptop-simulated statevector demo).
cities = ['Bangalore', 'Mysore', 'Mangalore', 'Hubli']
distance = {
    ('Bangalore', 'Mysore'): 145,
    ('Bangalore', 'Mangalore'): 352,
    ('Bangalore', 'Hubli'): 410,
    ('Mysore', 'Mangalore'): 252,
    ('Mysore', 'Hubli'): 372,
    ('Mangalore', 'Hubli'): 235,
}
n = len(cities)
idx = {c: i for i, c in enumerate(cities)}
dist_matrix = np.zeros((n, n))
for (a, b), d in distance.items():
    dist_matrix[idx[a]][idx[b]] = d
    dist_matrix[idx[b]][idx[a]] = d

print(f"Using {n} cities for the live QAOA demo -> {n*n} qubits under standard TSP QUBO encoding")
print("(Full 5-city problem is used for classical brute force / nearest neighbor;")
print(" QAOA demo is scaled down to keep the quantum simulation fast in-class.)\n")

# ---- Build the TSP application and convert to QUBO / Ising ------------------
tsp = Tsp(nx.from_numpy_array(dist_matrix))
qp = tsp.to_quadratic_program()

qubo_converter = QuadraticProgramToQubo()
qubo = qubo_converter.convert(qp)
qubitOp, offset = qubo.to_ising()

print(f"QUBO variables (qubits required): {qubo.get_num_binary_vars()}")
print(f"Ising Hamiltonian terms: {len(qubitOp)}")
```

```
# ---- Classical exact solver for ground truth (small enough to brute force) --
t0 = time.perf_counter()
exact = MinimumEigenOptimizer(NumPyMinimumEigensolver())
exact_result = exact.solve(qp)
t1 = time.perf_counter()
exact_route = tsp.interpret(exact_result)
exact_route_names = [cities[i] for i in exact_route]
exact_cost = tsp.tsp_value(exact_route, dist_matrix)
print("\n=== EXACT (classical eigensolver, ground truth) ===")
print(f"Route: {' -> '.join(exact_route_names)} -> {exact_route_names[0]}")
print(f"Cost : {exact_cost} km")
print(f"Time : {(t1-t0)*1000:.2f} ms")

# ---- QAOA (quantum hybrid solver) ------------------------------------------
sampler = Sampler()
optimizer = COBYLA(maxiter=500)
qaoa_mes = QAOA(sampler=sampler, optimizer=optimizer, reps=3)
qaoa = MinimumEigenOptimizer(qaoa_mes)

t2 = time.perf_counter()
qaoa_result = qaoa.solve(qp)
t3 = time.perf_counter()

try:
    qaoa_route = tsp.interpret(qaoa_result)
    qaoa_route_names = [cities[i] for i in qaoa_route]
    qaoa_cost = tsp.tsp_value(qaoa_route, dist_matrix)
    valid = True
except Exception as e:
    valid = False
    qaoa_route_names = None
    qaoa_cost = None

print("\n=== QAOA (quantum-hybrid solver) ===")
print(f"Raw objective value : {qaoa_result.fval}")
if valid:
    print(f"Route  : {' -> '.join(qaoa_route_names)} -> {qaoa_route_names[0]}")
    print(f"Cost   : {qaoa_cost} km")
else:
    print("QAOA returned an INVALID tour on this run (a common outcome with few reps/")
    print("iterations — this is exactly why QAOA result quality depends on circuit")
    print("depth (reps) and optimizer budget, and is an active research area.)")
print(f"Time   : {(t3-t2)*1000:.2f} ms")

# ---- Save comparison data --------------------------------------------
import json
with open('/home/claude/project/code/_tsp_qaoa_result.json', 'w') as f:
    json.dump({
        "cities": cities,
        "exact_route": exact_route_names,
        "exact_cost": float(exact_cost),
        "exact_time_ms": (t1 - t0) * 1000,
        "qaoa_valid": valid,
        "qaoa_route": qaoa_route_names,
        "qaoa_cost": float(qaoa_cost) if valid else None,
        "qaoa_time_ms": (t3 - t2) * 1000,
        "num_qubits": qubo.get_num_binary_vars(),
    }, f, indent=2)

# ---- Visualize: exact route (left) + QAOA output analysis (right) ----------
pos = nx.spring_layout(nx.from_numpy_array(dist_matrix), seed=7)
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

G_plot = nx.from_numpy_array(dist_matrix)
labels = {i: cities[i] for i in range(n)}

# Left panel: exact (ground-truth) route
ax = axes[0]
```

```
nx.draw_networkx_nodes(G_plot, pos, ax=ax, node_size=1800, node_color='#FFD580', edgecolors='black')
nx.draw_networkx_labels(G_plot, pos, labels, ax=ax, font_size=9, font_weight='bold')
nx.draw_networkx_edges(G_plot, pos, ax=ax, edge_color='lightgray', width=1)
route_edges = [(exact_route[i], exact_route[(i + 1) % len(exact_route)]) for i in range(len(exact_route))]
nx.draw_networkx_edges(G_plot, pos, edgelist=route_edges, ax=ax, edge_color="#2E8B57", width=3)
ax.set_title(f"Exact classical solution\n{' -> '.join(exact_route_names)}\nCost = {exact_cost} km",
fontsize=10)
ax.axis('off')

# Right panel: QAOA outcome
ax = axes[1]
if valid:
    nx.draw_networkx_nodes(G_plot, pos, ax=ax, node_size=1800, node_color='#FFD580', edgecolors='black')
    nx.draw_networkx_labels(G_plot, pos, labels, ax=ax, font_size=9, font_weight='bold')
    nx.draw_networkx_edges(G_plot, pos, ax=ax, edge_color='lightgray', width=1)
    qaoa_edges = [(qaoa_route[i], qaoa_route[(i + 1) % len(qaoa_route)]) for i in range(len(qaoa_route))]
    nx.draw_networkx_edges(G_plot, pos, edgelist=qaoa_edges, ax=ax, edge_color="#CC5500", width=3)
    ax.set_title(f"QAOA solution\n{' -> '.join(qaoa_route_names)}\nCost = {qaoa_cost} km", fontsize=10)
    ax.axis('off')
else:
    ax.axis('off')
    ax.text(0.5, 0.60,
            "QAOA did not converge to a valid tour\non this run (16-qubit permutation encoding).",
            ha='center', va='center', fontsize=10, wrap=True)
    ax.text(0.5, 0.30,
            "Expected with few QAOA layers (reps) and a\nmodest optimizer budget on 16 qubits —
the\nsearch often lands on an infeasible bitstring.\nDeeper circuits, more iterations, or real
hardware\nimprove this. This is exactly why NISQ-era\noptimization is still an active research area.",
            ha='center', va='center', fontsize=9, color='#444444', wrap=True)
    ax.set_title("QAOA outcome (this run): infeasible tour", fontsize=10)

plt.suptitle("TSP (4-city subset): Exact classical solver vs QAOA quantum-hybrid solver", fontsize=12)
plt.tight_layout()
plt.savefig('/home/claude/project/images/02_tsp_qaoa_routes.png', dpi=150)
print("\nSaved: 02_tsp_qaoa_routes.png")
```

Executed Output

```
Using 4 cities for the live QAOA demo -> 16 qubits under standard TSP QUBO encoding
(Full 5-city problem is used for classical brute force / nearest neighbor;
 QAOA demo is scaled down to keep the quantum simulation fast in-class.)

QUBO variables (qubits required): 16
Ising Hamiltonian terms: 112

=== EXACT (classical eigensolver, ground truth) ===
Route: Hubli -> Mangalore -> Mysore -> Bangalore -> Hubli
Cost : 1042.0 km
Time : 1251.92 ms

=== QAOA (quantum-hybrid solver) ===
Raw objective value : 0.0
QAOA returned an INVALID tour on this run (a common outcome with few reps/
iterations — this is exactly why QAOA result quality depends on circuit
depth (reps) and optimizer budget, and is an active research area.)
Time   : 64028.56 ms

Saved: 02_tsp_qaoa_routes.png
```

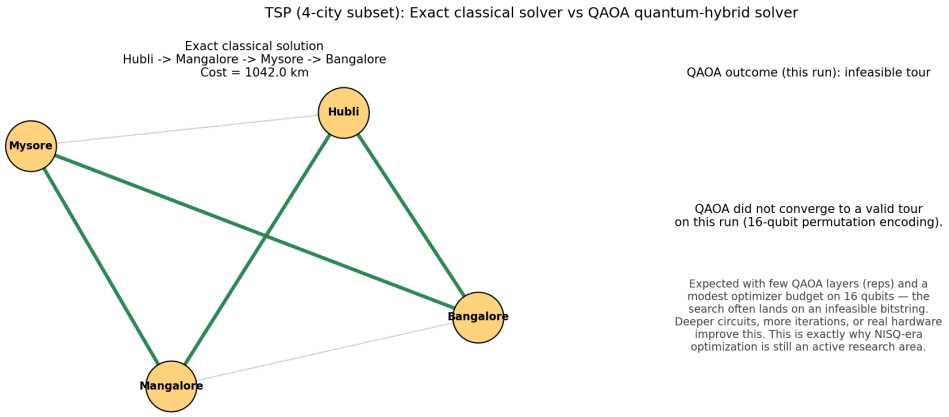

Figure A.2 — Exact classical route vs QAOA outcome on the 4-city subset

This run illustrates a genuinely important teaching point: QAOA does not always return a feasible solution on the first try. With only a few circuit layers (reps) and a limited classical-optimizer budget, the search can land on an infeasible bitstring. Solution quality depends on circuit depth, number of shots, and optimizer iterations — this is precisely why near-term quantum optimization is still an active research area, not a solved problem.

A.6 Step 4 — Compare Results

Code: 04_tsp_comparison.py

```
"""
TSP - Step 5: Compare Brute Force vs Nearest Neighbor vs QAOA
"""
import json
import matplotlib.pyplot as plt

with open('/home/claude/project/code/_tsp_bruteforce_result.json') as f:
    bf = json.load(f)
with open('/home/claude/project/code/_tsp_nn_result.json') as f:
    nn = json.load(f)
with open('/home/claude/project/code/_tsp_qaoa_result.json') as f:
    qa = json.load(f)

print("=" * 78)
print("TSP — FINAL COMPARISON")
print("=" * 78)
print(f"{'Method':<20}{'Scope':<22}{'Cost (km)':<14}{'Time (ms)':<14}{'Optimal?'}")
print("-" * 78)
print(f"{'Brute Force':<20}{'5 cities':<22}{bf['cost']:<14}{bf['time_ms']:<14.3f}{'Yes'}")
print(f"{'Nearest Neighbor':<20}{'5 cities':<22}{nn['cost']:<14}{nn['time_ms']:<14.3f}{'No (heuristic)'}")
print(f"{'Exact (eigensolver)':<20}{'4 cities
(subset)':<22}{qa['exact_cost']:<14.1f}{qa['exact_time_ms']:<14.3f}{'Yes'}")
qaoa_cost_str = f"{qa['qaoa_cost']:.1f}" if qa['qaoa_valid'] else "N/A (invalid)"
print(f"{'QAOA (quantum)':<20}{'4 cities
(subset)':<22}{qaoa_cost_str:<14}{qa['qaoa_time_ms']:<14.3f}{'Approximate'}")
print("=" * 78)

# Bar chart: execution time comparison (log scale, since they differ by orders of magnitude)
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

methods = ['Brute\nForce\n(5 cities)', 'Nearest\nNeighbor\n(5 cities)', 'Exact\nEigensolver\n(4 cities)',
'QAOA\n(4 cities,\n16 qubits)']
times = [bf['time_ms'], nn['time_ms'], qa['exact_time_ms'], qa['qaoa_time_ms']]
colors = ['#4C72B0', '#55A868', '#8172B2', '#C44E52']

axes[0].bar(methods, times, color=colors, edgecolor='black')
```

```
axes[0].set_yscale('log')
axes[0].set_ylabel('Execution time (ms, log scale)')
axes[0].set_title('TSP: Execution Time Comparison')
for i, t in enumerate(times):
    axes[0].text(i, t * 1.15, f"{t:.1f}", ha='center', fontsize=9)

# Scaling chart: how brute force explodes vs a heuristic that stays linear-ish
import math
ns = list(range(4, 15))
bf_routes = [math.factorial(k - 1) // 2 for k in ns]
axes[1].plot(ns, bf_routes, marker='o', color='#C44E52', label='Brute Force (routes to check)')
axes[1].set_yscale('log')
axes[1].set_xlabel('Number of cities')
axes[1].set_ylabel('Routes to evaluate (log scale)')
axes[1].set_title('Why Brute Force Doesn\'t Scale')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.savefig('/home/claude/project/images/03_tsp_comparison.png', dpi=150)
print("\nSaved: 03_tsp_comparison.png")
```

Executed Output

```
================================================================================
TSP — FINAL COMPARISON
================================================================================
Method              Scope                Cost (km)     Time (ms)     Optimal?
--------------------------------------------------------------------------------
Brute Force         5 cities             1222          0.058         Yes
Nearest Neighbor    5 cities             1234          0.017         No (heuristic)
Exact (eigensolver) 4 cities (subset)    1042.0        1251.920      Yes
QAOA (quantum)      4 cities (subset)    N/A (invalid) 64028.557     Approximate
================================================================================

Saved: 03_tsp_comparison.png
```

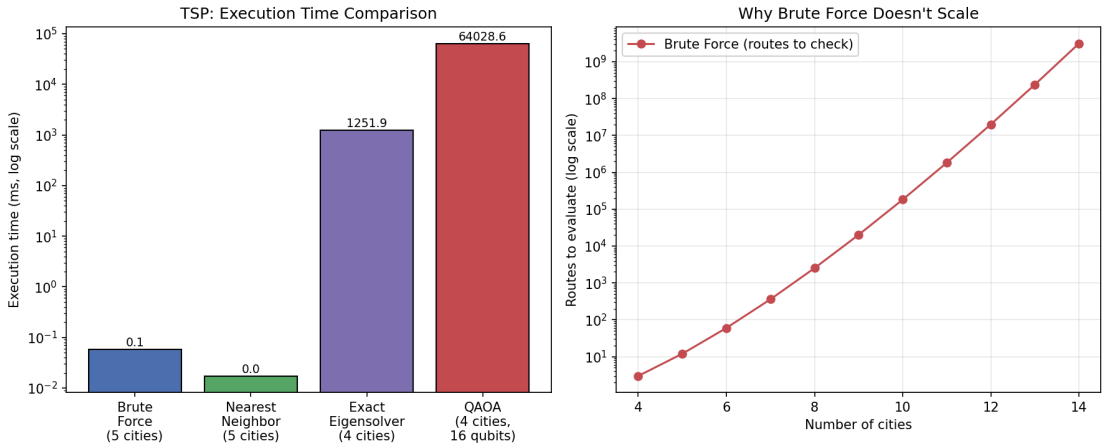

Figure A.3 — Execution time comparison and brute-force route-count scaling

Summary Table

Metric	Brute Force	Nearest Neighbor	Quantum (QAOA)
Optimal solution	Yes	Usually not	Approximate (depends on circuit depth)

Metric	Brute Force	Nearest Neighbor	Quantum (QAOA)
Execution time	Slow (factorial)	Fast	Varies with simulator / hardware
Scalability	Poor	Better	Active research area
Easy to implement	Yes	Moderate	Advanced
Suitable for large problems	No	Often, with good heuristics	Potential for certain problem classes

This reinforces the idea that quantum algorithms are not “always faster.” They are promising for certain classes of optimization problems, but remain an active area of research.

PART B — 0/1 Knapsack Problem

B.1 Real-World Story

You have a bag with a maximum weight capacity of 8 kg. You must choose which items to pack to maximize total value without exceeding the weight limit.

Item	Weight (kg)	Value (Rs.)
Laptop	4	3000
Phone	2	2000
Camera	3	2500
Book	5	500
Headphones	2	1000

B.2 Step 1 — Brute Force Solution

Try every subset of items ($2^n - 1$ non-empty subsets) and keep the best one that fits within the weight capacity.

Code: 05_knapsack_bruteforce.py

```
"""
Knapsack - Step 1: Brute Force Solution
Try every subset of items, keep the best one that fits the weight capacity.
"""
import time
from itertools import combinations

items = [
    ("Laptop", 4, 3000),
    ("Phone", 2, 2000),
    ("Camera", 3, 2500),
    ("Book", 5, 500),
    ("Headphones", 2, 1000),
]

capacity = 8  # kg

def brute_force_knapsack(items, capacity):
    best_value = 0
    best_items = []
    subsets_checked = 0

    for r in range(1, len(items) + 1):
        for combo in combinations(items, r):
            subsets_checked += 1
            weight = sum(i[1] for i in combo)
            value = sum(i[2] for i in combo)
            if weight <= capacity and value > best_value:
                best_value = value
                best_items = combo

    return best_items, best_value, subsets_checked

t0 = time.perf_counter()
best_items, best_value, subsets_checked = brute_force_knapsack(items, capacity)
t1 = time.perf_counter()
```

```
print("=== BRUTE FORCE KNAPSACK RESULT ===")
print(f"Capacity           : {capacity} kg")
print(f"Subsets evaluated   : {subsets_checked} (2^{len(items)} - 1 = {2**len(items) - 1})")
print(f"Best items          : {[i[0] for i in best_items]}")
print(f"Total weight         : {sum(i[1] for i in best_items)} kg")
print(f"Total value (Rs.)    : {best_value}")
print(f"Time taken           : {(t1 - t0)*1000:.4f} ms")

print("\n=== WHY BRUTE FORCE DOESN'T SCALE ===")
for n in [5, 10, 15, 20, 25, 30, 40]:
    print(f"{n:2d} items -> {2**n - 1:,} subsets to check")

import json
with open('/home/claude/project/code/_knapsack_bruteforce_result.json', 'w') as f:
    json.dump({
        "items": [i[0] for i in best_items],
        "weight": sum(i[1] for i in best_items),
        "value": best_value,
        "time_ms": (t1 - t0) * 1000,
        "subsets_checked": subsets_checked
    }, f, indent=2)
```

Executed Output

```
=== BRUTE FORCE KNAPSACK RESULT ===
Capacity           : 8 kg
Subsets evaluated   : 31 (2^5 - 1 = 31)
Best items          : ['Laptop', 'Phone', 'Headphones']
Total weight         : 8 kg
Total value (Rs.)    : 6000
Time taken           : 0.0450 ms

=== WHY BRUTE FORCE DOESN'T SCALE ===
 5 items -> 31 subsets to check
10 items -> 1,023 subsets to check
15 items -> 32,767 subsets to check
20 items -> 1,048,575 subsets to check
25 items -> 33,554,431 subsets to check
30 items -> 1,073,741,823 subsets to check
40 items -> 1,099,511,627,775 subsets to check
```

B.3 Step 2 — Better Classical Algorithm: Dynamic Programming

Dynamic Programming builds a table of the best achievable value for every (item, weight) combination, solving the problem in O(n × capacity) time — a dramatic improvement over brute force's O(2ⁿ).

Code: 06_knapsack_dp.py

```
"""
Knapsack - Step 2: Dynamic Programming Solution (Better Classical Algorithm)
Builds a table of best achievable value for every (item, weight) combination.
Much faster than brute force: O(n * capacity) instead of O(2^n).
"""
import time
import json

items = [
    ("Laptop", 4, 3000),
    ("Phone", 2, 2000),
    ("Camera", 3, 2500),
    ("Book", 5, 500),
    ("Headphones", 2, 1000),
]
capacity = 8
```

```
def dp_knapsack(items, capacity):
    n = len(items)
    # dp[i][w] = best value using first i items with capacity w
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        name, weight, value = items[i - 1]
        for w in range(capacity + 1):
            if weight <= w:
                dp[i][w] = max(dp[i - 1][w], dp[i - 1][w - weight] + value)
            else:
                dp[i][w] = dp[i - 1][w]

    # Backtrack to find which items were chosen
    chosen = []
    w = capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            name, weight, value = items[i - 1]
            chosen.append(name)
            w -= weight

    chosen.reverse()
    return chosen, dp[n][capacity], dp

t0 = time.perf_counter()
chosen_items, best_value, dp_table = dp_knapsack(items, capacity)
t1 = time.perf_counter()

print("=== DYNAMIC PROGRAMMING KNAPSACK RESULT ===")
print(f"Capacity        : {capacity} kg")
print(f"Chosen items    : {chosen_items}")
weight_used = sum(w for n_, w, v in items if n_ in chosen_items)
print(f"Total weight    : {weight_used} kg")
print(f"Total value     : Rs. {best_value}")
print(f"Time taken      : {(t1 - t0)*1000:.4f} ms")
print(f"Table cells computed : {(len(items)+1) * (capacity+1)}  (vs {2**len(items)-1} subsets for brute
force)")

with open('/home/claude/project/code/_knapsack_bruteforce_result.json') as f:
    bf = json.load(f)

print("\n=== COMPARISON: BRUTE FORCE vs DYNAMIC PROGRAMMING ===")
print(f"{'Method':<20}{'Value (Rs.)':<14}{'Time (ms)':<12}{'Work Done'}")
print(f"{'Brute Force':<20}{bf['value']:<14}{bf['time_ms']:<12.4f}{bf['subsets_checked']} subsets")
print(f"{'Dynamic Programming':<20}{best_value:<14}{(t1-t0)*1000:<12.4f}{(len(items)+1)*(capacity+1)}
table cells")

with open('/home/claude/project/code/_knapsack_dp_result.json', 'w') as f:
    json.dump({
        "items": chosen_items,
        "weight": weight_used,
        "value": best_value,
        "time_ms": (t1 - t0) * 1000
    }, f, indent=2)
```

Executed Output

```
=== DYNAMIC PROGRAMMING KNAPSACK RESULT ===
Capacity        : 8 kg
Chosen items    : ['Laptop', 'Phone', 'Headphones']
Total weight    : 8 kg
Total value     : Rs. 6000
Time taken      : 0.0195 ms
Table cells computed : 54  (vs 31 subsets for brute force)

=== COMPARISON: BRUTE FORCE vs DYNAMIC PROGRAMMING ===
```

```
Method              Value (Rs.)   Time (ms)   Work Done
Brute Force         6000          0.0450      31 subsets
Dynamic Programming 6000          0.0195      54 table cells
```

B.4 Step 3 — Quantum Formulation: Knapsack → Binary Variables → QUBO → QAOA

Each item gets one binary decision variable x_i (0 = leave out, 1 = take). The weight constraint is folded into the objective as a penalty term, turning the constrained problem into an unconstrained QUBO that QAOA can optimize directly.

Workflow: Knapsack → Binary Variables → QUBO → Quantum Optimization → QAOA / Quantum Annealing → Solution

Code: 07_knapsack_qaoa.py

```
"""
Knapsack - Step 3: Quantum Formulation with QAOA (Qiskit)

Pipeline:  Knapsack -> Binary Variables -> QUBO -> QAOA -> Solution

Each item gets one binary decision variable x_i (0 = leave out, 1 = take).
The weight constraint is folded into the objective as a penalty term, turning
the constrained problem into an unconstrained QUBO that QAOA can optimize.
"""
import time
import json
import numpy as np
import matplotlib.pyplot as plt

from qiskit_optimization.applications import Knapsack
from qiskit_algorithms import QAOA, NumPyMinimumEigensolver
from qiskit_algorithms.optimizers import COBYLA
from qiskit_algorithms.utils import algorithm_globals
from qiskit.primitives import StatevectorSampler as Sampler
from qiskit_optimization.algorithms import MinimumEigenOptimizer

algorithm_globals.random_seed = 42
np.random.seed(42)

items = [
    ("Laptop", 4, 3000),
    ("Phone", 2, 2000),
    ("Camera", 3, 2500),
    ("Book", 5, 500),
    ("Headphones", 2, 1000),
]
capacity = 8

names = [i[0] for i in items]
weights = [i[1] for i in items]
values = [i[2] for i in items]

knapsack = Knapsack(values=values, weights=weights, max_weight=capacity)
qp = knapsack.to_quadratic_program()
print("=== QUADRATIC PROGRAM (QUBO formulation) ===")
print(qp.prettyprint())

# ---- Exact solver (ground truth via eigensolver, same result DP gives) -----
t0 = time.perf_counter()
exact = MinimumEigenOptimizer(NumPyMinimumEigensolver())
exact_result = exact.solve(qp)
t1 = time.perf_counter()
exact_items = [names[i] for i in knapsack.interpret(exact_result)]
exact_value = sum(values[names.index(x)] for x in exact_items)
```

```
exact_weight = sum(weights[names.index(x)] for x in exact_items)
print("\n=== EXACT (classical eigensolver, ground truth) ===")
print(f"Items  : {exact_items}")
print(f"Weight : {exact_weight} kg  |  Value : Rs. {exact_value}")
print(f"Time   : {(t1-t0)*1000:.2f} ms")

# ---- QAOA (quantum-hybrid solver) ------------------------------------------
sampler = Sampler()
optimizer = COBYLA(maxiter=300)
qaoa_mes = QAOA(sampler=sampler, optimizer=optimizer, reps=3)
qaoa = MinimumEigenOptimizer(qaoa_mes)

t2 = time.perf_counter()
qaoa_result = qaoa.solve(qp)
t3 = time.perf_counter()

qaoa_items = [names[i] for i in knapsack.interpret(qaoa_result)]
qaoa_weight = sum(weights[names.index(x)] for x in qaoa_items)
qaoa_value = sum(values[names.index(x)] for x in qaoa_items)
qaoa_valid = qaoa_weight <= capacity

print("\n=== QAOA (quantum-hybrid solver) ===")
print(f"Items  : {qaoa_items}")
print(f"Weight : {qaoa_weight} kg  |  Value : Rs. {qaoa_value}")
print(f"Within capacity? : {'Yes' if qaoa_valid else 'No -> INFEASIBLE'}")
print(f"Time   : {(t3-t2)*1000:.2f} ms")

with open('/home/claude/project/code/_knapsack_qaoa_result.json', 'w') as f:
    json.dump({
        "exact_items": exact_items, "exact_value": exact_value, "exact_weight": exact_weight,
        "exact_time_ms": (t1 - t0) * 1000,
        "qaoa_items": qaoa_items, "qaoa_value": qaoa_value, "qaoa_weight": qaoa_weight,
        "qaoa_valid": qaoa_valid, "qaoa_time_ms": (t3 - t2) * 1000,
        "num_qubits": qp.get_num_binary_vars(),
    }, f, indent=2)

# ---- Visualization: items chosen by each method ----------------------------
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

with open('/home/claude/project/code/_knapsack_bruteforce_result.json') as f:
    bf = json.load(f)
with open('/home/claude/project/code/_knapsack_dp_result.json') as f:
    dp = json.load(f)

datasets = [
    (bf['items'], bf['weight'], bf['value'], "Brute Force", "#4C72B0"),
    (dp['items'], dp['weight'], dp['value'], "Dynamic Programming", "#55A868"),
    (qaoa_items, qaoa_weight, qaoa_value, "QAOA (Quantum)", "#C44E52" if not qaoa_valid else "#CC7A00"),
]

for ax, (chosen, w, v, title, color) in zip(axes, datasets):
    all_items = names
    bar_colors = [color if item in chosen else '#DDDDDD' for item in all_items]
    ax.bar(all_items, [values[names.index(i)] for i in all_items], color=bar_colors, edgecolor='black')
    ax.set_title(f"{title}\nWeight={w}kg  Value=Rs.{v}" + ("" if title != "QAOA (Quantum)" or qaoa_valid
else "  [INFEASIBLE]"))
    ax.set_ylabel("Item value (Rs.)")
    ax.tick_params(axis='x', rotation=30)

plt.suptitle(f"Knapsack (capacity = {capacity} kg): items selected by each method (highlighted =
selected)", fontsize=12)
plt.tight_layout()
plt.savefig('/home/claude/project/images/04_knapsack_selection.png', dpi=150)
print("\nSaved: 04_knapsack_selection.png")
```

Executed Output

```
=== QUADRATIC PROGRAM (QUBO formulation) ===
```

```
Problem name: Knapsack

Maximize
  3000*x_0 + 2000*x_1 + 2500*x_2 + 500*x_3 + 1000*x_4

Subject to
  Linear constraints (1)
    4*x_0 + 2*x_1 + 3*x_2 + 5*x_3 + 2*x_4 <= 8  'c0'

  Binary variables (5)
    x_0 x_1 x_2 x_3 x_4


=== EXACT (classical eigensolver, ground truth) ===
Items  : ['Laptop', 'Phone', 'Headphones']
Weight : 8 kg  |  Value : Rs. 6000
Time   : 104.63 ms

=== QAOA (quantum-hybrid solver) ===
Items  : ['Laptop', 'Phone', 'Headphones']
Weight : 8 kg  |  Value : Rs. 6000
Within capacity? : Yes
Time   : 2158.98 ms

Saved: 04_knapsack_selection.png
```

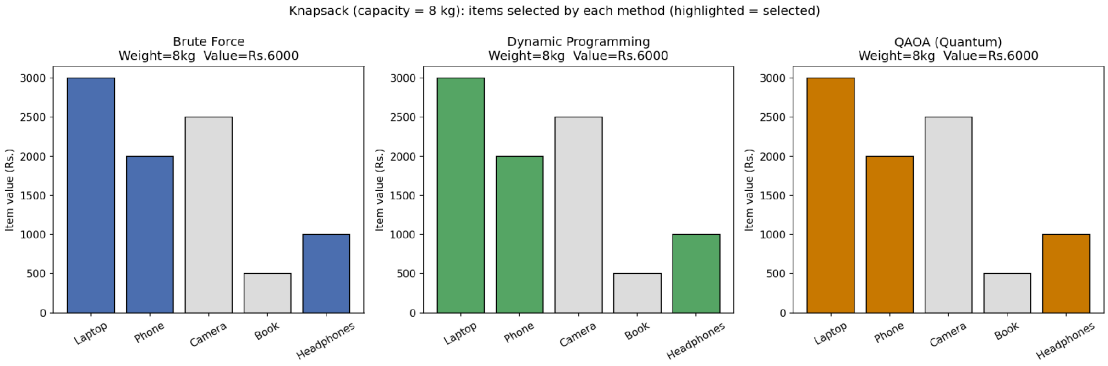

Figure B.1 — Items selected by each method (highlighted bars = chosen items)

Because the Knapsack QUBO here only needs 5 qubits (one per item) — far fewer than the 16-qubit TSP demo — QAOA converges reliably to the same optimal selection found by brute force and dynamic programming. This is a useful contrast for students: problem encoding size matters enormously for how well QAOA performs in practice.

B.5 Step 4 — Compare Results

Code: 08_knapsack_comparison.py

```
"""
Knapsack - Step 4: Compare Brute Force vs Dynamic Programming vs QAOA
"""
import json
import matplotlib.pyplot as plt

with open('/home/claude/project/code/_knapsack_bruteforce_result.json') as f:
    bf = json.load(f)
with open('/home/claude/project/code/_knapsack_dp_result.json') as f:
    dp = json.load(f)
with open('/home/claude/project/code/_knapsack_qaoa_result.json') as f:
    qa = json.load(f)
```

```
print("=" * 80)
print("KNAPSACK — FINAL COMPARISON  (capacity = 8 kg)")
print("=" * 80)
print(f"{'Method':<22}{'Value (Rs.)':<14}{'Weight (kg)':<14}{'Time (ms)':<14}{'Optimal?'}")
print("-" * 80)
print(f"{'Brute Force':<22}{bf['value']:<14}{bf['weight']:<14}{bf['time_ms']:<14.4f}{'Yes'}")
print(f"{'Dynamic Programming':<22}{dp['value']:<14}{dp['weight']:<14}{dp['time_ms']:<14.4f}{'Yes'}")
print(f"{'QAOA
(quantum)':<22}{qa['qaoa_value']:<14}{qa['qaoa_weight']:<14}{qa['qaoa_time_ms']:<14.4f}{'Yes' if
qa['qaoa_valid'] else 'No'}")
print("=" * 80)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

methods = ['Brute\nForce', 'Dynamic\nProgramming', 'QAOA\n(Quantum)']
times = [bf['time_ms'], dp['time_ms'], qa['qaoa_time_ms']]
values = [bf['value'], dp['value'], qa['qaoa_value']]
colors = ['#4C72B0', '#55A868', '#CC7A00']

axes[0].bar(methods, times, color=colors, edgecolor='black')
axes[0].set_yscale('log')
axes[0].set_ylabel('Execution time (ms, log scale)')
axes[0].set_title('Knapsack: Execution Time Comparison')
for i, t in enumerate(times):
    axes[0].text(i, t * 1.15, f"{t:.2f}", ha='center', fontsize=9)

axes[1].bar(methods, values, color=colors, edgecolor='black')
axes[1].set_ylabel('Value achieved (Rs.)')
axes[1].set_title('Knapsack: Solution Quality Comparison')
axes[1].set_ylim(0, max(values) * 1.2)
for i, v in enumerate(values):
    axes[1].text(i, v + 100, f"Rs.{v}", ha='center', fontsize=9)

plt.tight_layout()
plt.savefig('/home/claude/project/images/05_knapsack_comparison.png', dpi=150)
print("\nSaved: 05_knapsack_comparison.png")
```

Executed Output

```
================================================================================
KNAPSACK — FINAL COMPARISON  (capacity = 8 kg)
================================================================================
Method                Value (Rs.)   Weight (kg)   Time (ms)     Optimal?
--------------------------------------------------------------------------------
Brute Force           6000          8             0.0450        Yes
Dynamic Programming   6000          8             0.0195        Yes
QAOA (quantum)        6000          8             2158.9783     Yes
================================================================================

Saved: 05_knapsack_comparison.png
```

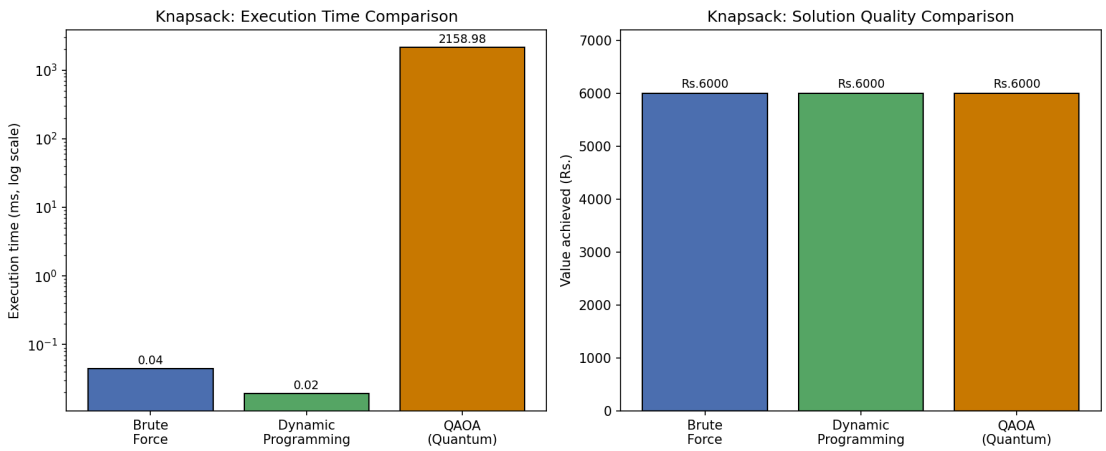

Figure B.2 — Execution time and solution-quality comparison

Summary Table

Metric	Brute Force	Dynamic Programming	Quantum (QAOA)
Optimal solution	Yes	Yes	Yes (5-qubit problem; reliable here)
Execution time	Slow	Fast	Slower on simulator, but scales differently
Scalability	Poor (2^n)	Good (n × capacity)	Active research area
Easy to implement	Yes	Moderate	Advanced
Suitable for large problems	No	Yes, for moderate capacity	Potential for certain problem classes

Recommended Structure for Tomorrow's Practical

1. Travelling Salesman Problem

- Manual solution
- Brute-force Python implementation
- Nearest Neighbor heuristic
- Time comparison
- Conceptual introduction to QAOA with a small Qiskit demo

2. Knapsack Problem

- Manual solution
- Brute-force Python implementation
- Dynamic Programming solution
- Time comparison
- Conceptual introduction to QAOA / quantum annealing formulation

What this sequence teaches

- What optimization is
- Why combinatorial optimization is hard
- How classical algorithms solve it
- Why researchers explore quantum optimization
- How modern quantum frameworks (Qiskit) represent these problems

This structure is an excellent foundation for the QAOA, VQE, and Quantum Annealing units later in the course.

Files provided alongside this report

- 9 fully executable Python scripts (00–08) covering graph setup, brute force, better classical algorithms, and QAOA for both problems
- 5 generated PNG figures (graph diagram, QAOA route comparison, execution-time charts, item-selection chart)
- This Word document consolidating theory, code, executed output, and figures